

Cyril MAZAURIC

TP3 -- Structures de controle, fonctions et pointeurs

TP3 -- Structures de controle, fonctions et pointeurs.....	1
Partie 1 -- Faire des choix : conditions.....	1
1.3 Exercice : convertisseur de notes.....	2
Partie 2 -- Repeter : les boucles.....	2
2.3 Exercice : table de multiplication.....	2
Partie 3 -- Les fonctions.....	2
3.3 Exercice : bibliotheque de fonctions.....	3
Partie 4 -- Tableaux et chaines.....	3
4.3 Exercice : statistiques de notes.....	3
Partie 5 -- Introduction aux pointeurs.....	3
5.4 Exercice : fonctions avec pointeurs.....	3
Partie 6 -- Mini-projet : devine le nombre.....	4

Partie 1 -- Faire des choix : conditions

1.3 Exercice : convertisseur de notes

Creez `notes.c` qui :

1. Demande une note sur 20
2. Affiche la mention correspondante (Tres bien, Bien, Assez bien, Passable, Insuffisant)
3. Affiche "Admis" si ≥ 10 , "Refuse" sinon
4. Gere les notes invalides (< 0 ou > 20)

Partie 2 -- Repeter : les boucles

2.3 Exercice : table de multiplication

Creez `table.c` qui :

1. Demande un nombre `n` a l'utilisateur
2. Affiche la table de multiplication de `n` (de 1 a 10)
3. Redemande un nombre tant que l'utilisateur ne saisit pas 0

Exemple de sortie :

```
Nombre (0 pour quitter) : 7
7 x 1  = 7
7 x 2  = 14
...
7 x 10 = 70
Nombre (0 pour quitter) : 0
Au revoir !
```

Partie 3 -- Les fonctions

3.3 Exercice : bibliotheque de fonctions

Creez `fonctions.c` avec les fonctions suivantes :

Fonction	Signature	Description
Minimum	<code>int minimum(int a, int b)</code>	Retourne le plus petit des deux
Maximum	<code>int maximum(int a, int b)</code>	Retourne le plus grand des deux
Valeur absolue	<code>int valeur_absolue(int x)</code>	Retourne la valeur absolue
Est pair	<code>int est_pair(int x)</code>	Retourne 1 si pair, 0 sinon
Factorielle	<code>long factorielle(int n)</code>	Calcule n! (iteratif avec une boucle)

Testez chaque fonction dans `main` avec `printf` .

Partie 4 -- Tableaux et chaines

4.3 Exercice : statistiques de notes

Creez `statistiques.c` qui :

1. Declare un tableau de 8 notes (entiers) initialisees en dur
2. Calcule et affiche : la somme, la moyenne, la note minimum et la note maximum
3. Utilisez des fonctions separees pour chaque calcul

Partie 5 -- Introduction aux pointeurs

5.4 Exercice : fonctions avec pointeurs

Creez `pointeurs.c` avec :

1. Une fonction `void min_max(int tab[], int n, int *pmin, int *pmax)` qui trouve le minimum et le maximum d'un tableau et les retourne via des pointeurs
2. Une fonction `double moyenne(int tab[], int n)` qui calcule la moyenne
3. Testez avec un tableau de 6 notes dans `main`

Partie 6 -- Mini-projet : devine le nombre

Creez `devine.c` : le jeu classique ou l'ordinateur tire un nombre au hasard et l'utilisateur doit le trouver.

Regles :

1. Inclure `stdlib.h` et `time.h`
2. Initialiser le generateur aleatoire : `srand(time(NULL));`
3. Tirer un nombre entre 1 et 100 : `rand() % 100 + 1`
4. Boucler tant que l'utilisateur n'a pas trouve : lire son essai, dire "trop petit" ou "trop grand"
5. Compter le nombre de tentatives et l'afficher a la fin
6. Encapsuler la logique de jeu dans une fonction `int jouer(int secret)` qui retourne le nombre de tentatives